
DBMS - Final Report

MarketPulse

A RAG Pipeline for Stock Price Movement Analysis

Submitted to:

Prof. – Prof. Innocent Nyalala

Course: Data Base Management System

Prepared by:

-> Nidheesh Deepu Nair
-> Shreyash Kumar Pandey
-> Anuj Gautam

Abstract

MarketPulse is an AI-integrated database system that explains stock price movements by linking structured financial data with news articles through a Retrieval-Augmented Generation (RAG) pipeline. The system ingests daily OHLCV price data for 50 publicly traded companies, news articles from four sources, analyst upgrade and downgrade records, and eight macroeconomic time series from the Federal Reserve, totalling roughly 89,000 rows across seven normalised tables. All data resides in a PostgreSQL 18 database with the pgvector extension, which stores 768-dimensional sentence embeddings alongside relational data in Third Normal Form. When a user poses a natural-language question, the system encodes it with the all-mpnet-base-v2 sentence-transformer model, retrieves semantically similar news through an HNSW cosine-similarity index, assembles a structured multi-table context, and forwards it to Google Gemini for a grounded, citation-backed answer. Targeted B-Tree and HNSW indexing, benchmarked with EXPLAIN ANALYZE, reduced buffer accesses by up to 73% and execution time by up to 3.3x on the system's core access patterns. The platform is accessible through a Streamlit web interface and a command-line tool, with a graceful fallback mode that returns raw context if the LLM is unavailable.

1. Introduction

Stock prices move for reasons that are rarely obvious from numbers alone. A five-percent intra-day swing in a technology stock might be driven by an earnings surprise, a regulatory announcement, a macroeconomic data release, or an analyst downgrade — often by a combination of these. Retail investors and finance students routinely face the question *"Why did this stock move?"* yet answering it requires cross-referencing price data, news, analyst opinion, and economic indicators in a way that no single existing tool provides in an integrated form.

MarketPulse addresses this gap. The system targets Track A of the Z2004 semester project: a question-answering system backed by a relational database and a vector store. The core goal is to accept natural-language questions such as *"What caused AAPL to drop this week?"* and return a grounded, source-cited answer that synthesises structured market data with semantically relevant news.

The project pursues four objectives: (1) design a normalised relational schema that faithfully models companies, sectors, prices, news, ratings, and macro indicators; (2) ingest real, non-synthetic data from public APIs and RSS feeds, clean it, score its sentiment, and generate vector embeddings; (3) demonstrate measurable query performance improvements through indexing with documented EXPLAIN ANALYZE evidence; and (4) wire these components into a working RAG pipeline accessible via web and CLI interfaces, with a fallback mode that guarantees output regardless of LLM availability.

2. System Architecture

MarketPulse follows a five-stage pipeline connecting user input to a grounded answer. A user submits a question through the Streamlit web interface or the CLI. The application first resolves a company context: if a ticker flag is supplied it is looked up directly; otherwise a heuristic scans the query for uppercase tokens of five characters or fewer and checks each against the companies table.

Second, the query string is encoded by the locally running all-mpnet-base-v2 model into a 768-dimensional vector capturing its semantic intent. Third, PostgreSQL retrieval runs in parallel: an HNSW cosine-similarity search over news_articles.embedding returns the most relevant articles (filtered through article_company when a company is known); B-Tree-indexed lookups fetch the five most recent prices and three most recent analyst ratings; and a window function (ROW_NUMBER partitioned by indicator_name) returns the latest value of each macroeconomic series. Fourth, all retrieved rows are assembled into one structured context string — company identity, prices, ratings, news with sentiment, then macro indicators. Fifth, this context, the question, and a citation-mandating system prompt are sent to Google Gemini.

If the LLM call fails — missing key, a retired model endpoint, or exhausted rate limits — the system enters fallback mode: all retrieval still executes normally and the raw structured context is returned directly, prefixed with a clear banner. The model candidate chain (gemini-2.5-flash → gemini-flash-latest → gemini-2.0-flash → gemini-2.5-pro), each retried up to three times with exponential back-off on rate-limit errors, ensures the system is resilient to individual model outages before falling back.

3. Database Schema Design

The MarketPulse schema comprises seven tables in PostgreSQL 18 with the pgvector extension enabled. PostgreSQL was selected over SQL Server specifically because the Track A guidelines require vector storage via pgvector or Pinecone, and keeping vectors and relational data in a single engine simplifies both query design and deployment. Figure 1 shows the complete entity-relationship diagram.

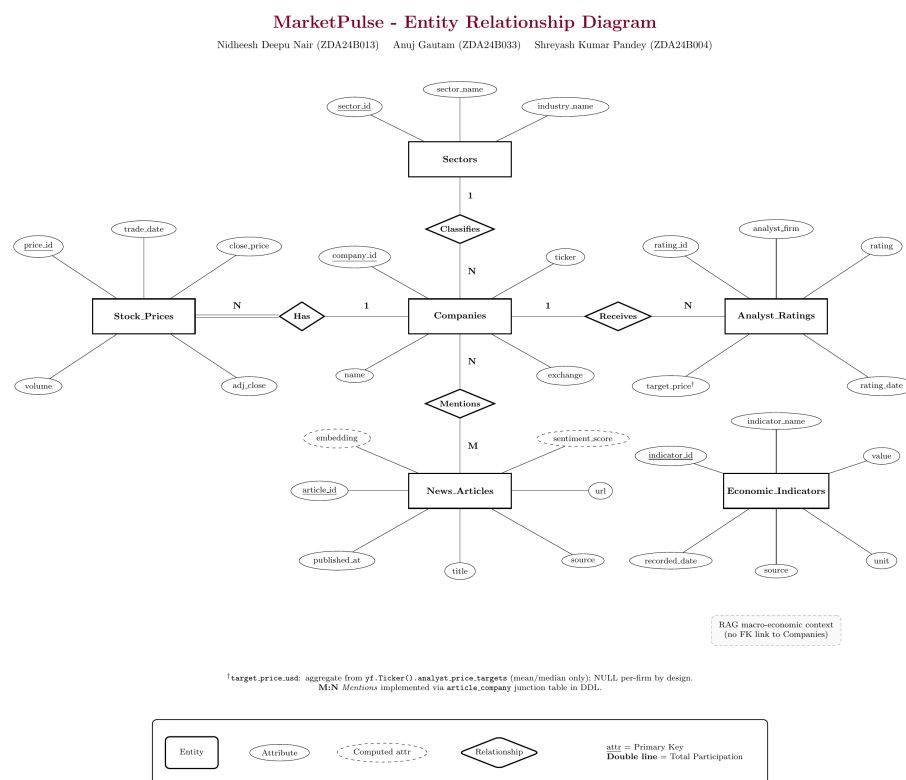


Figure 1. MarketPulse Entity-Relationship Diagram — seven entities, two M:N relationships resolved via junction tables, and a standalone macroeconomic context table.

3.1 Table Overview

Table	Purpose	Final Rows
sectors	GICS sector and industry classification lookup	24
companies	50 publicly traded companies with ticker, exchange, market cap	50
stock_prices	Daily OHLCV + adjusted close per company (2-year history)	25,050
news_articles	Title, content, sentiment score, 768-dim embedding per article	16,429
article_company	Junction table resolving the M:N articles-to-companies link	16,858
analyst_ratings	Analyst firm upgrades/downgrades with rating and date	27,214
economic_indicators	8 FRED macroeconomic time series (2022–present)	3,598

Table 1. The seven MarketPulse tables and their row counts in the final, fully ingested dataset — approximately 89,200 rows in total, well above the 1,000-row Track A minimum.

3.2 Third Normal Form Argument

All tables satisfy Third Normal Form. 1NF holds because every column stores a single atomic value with no repeating groups. 2NF holds because every non-key attribute depends on the whole

primary key — relevant for `article_company`, whose composite key (`article_id`, `company_id`) carries no further attributes that could exhibit partial dependency. 3NF holds because no transitive dependencies exist among non-key attributes. The key normalisation decision was decomposing sector and industry information out of companies into a dedicated sectors table: without this, `company_id` $\rightarrow$ `sector_name`, `industry_name` would be transitive through `sector_id`. Isolating sectors with its own surrogate key and a UNIQUE constraint on (`sector_name`, `industry_name`) removes this transitive dependency entirely. Full functional dependencies for all seven tables are documented in the `schema.sql` header comments and in the supplementary 3NF justification notes.

3.3 Constraints

Every table has a surrogate SERIAL primary key. Foreign keys link companies $\rightarrow$ sectors (ON DELETE RESTRICT), stock_prices and analyst_ratings $\rightarrow$ companies (ON DELETE CASCADE), and article_company $\rightarrow$ both news_articles and companies (ON DELETE CASCADE). UNIQUE constraints enforce candidate keys: ticker in companies; url in news_articles, which prevents duplicate ingestion of the same article across sources; (`company_id`, `trade_date`) in stock_prices; (`company_id`, `analyst_firm`, `rating_date`) in analyst_ratings; and (`indicator_name`, `recorded_date`) in economic_indicators. CHECK constraints enforce domain rules: positive prices, $\text{high} \geq \text{low}$, non-negative volume, sentiment_score bounded to $[-1.0, 1.0]$, ratings limited to a nine-value controlled vocabulary, positive target prices where present, exchange limited to six recognised values, and non-negative market capitalisation.

4. Data Sources and Dataset

Source	Data Provided	Access
yfinance	OHLCV prices, company metadata, analyst upgrades/downgrades	Python library, no key
NewsAPI.org	News title/description/url/date (content truncated, free tier)	API key, 100 req/day
GNews API	Supplementary news corpus	API key, free tier
Yahoo Finance RSS	Financial news via feedparser	No key required
Google News RSS	Financial news via feedparser	No key required
FRED (fredapi)	8 macroeconomic series, 2022–present	Free API key

Table 2. Data sources feeding the MarketPulse ingestion pipeline.

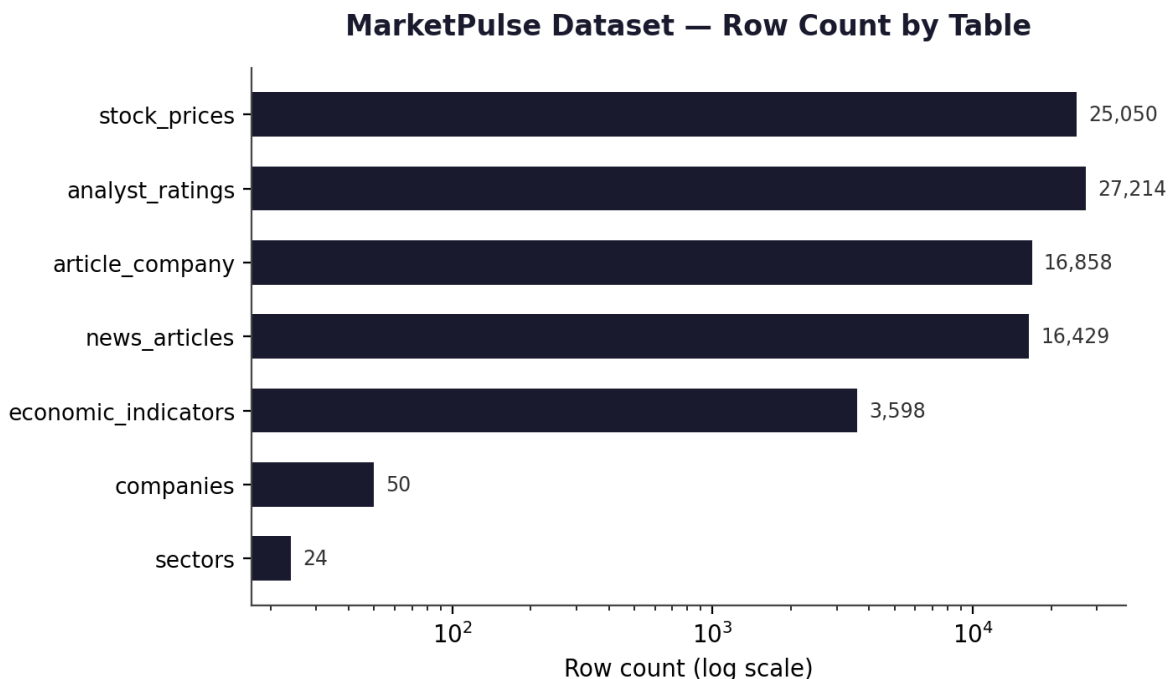


Figure 2. Final row count per table (log scale). *stock_prices*, *analyst_ratings*, *news_articles*, and *article_company* dominate the dataset by volume.

4.1 Cleaning, Sentiment, and Embeddings

Sentiment scoring uses NLTK's VADER analyser, run locally with no external API, producing a compound score from -1.0 to 1.0 from each article's title and description before insertion. A known limitation of the NewsAPI free tier is that article content is truncated to roughly 200 characters; this was handled by generating both sentiment scores and embeddings from (title + description) rather than the content column, since these fields are always complete. Embeddings use all-mpnet-base-v2 from sentence-transformers, producing 768-dimensional vectors stored in pgvector's VECTOR(768) type and indexed with HNSW for approximate nearest-neighbour search. The url column serves as the deduplication key across all four news sources, enforced by a UNIQUE constraint.

5. SQL Queries

queries.sql contains 14 labelled queries demonstrating the required SQL feature set, several intentionally satisfying more than one category. Figure 3 shows coverage against the rubric's minimum of two queries per category; every category clears the bar.

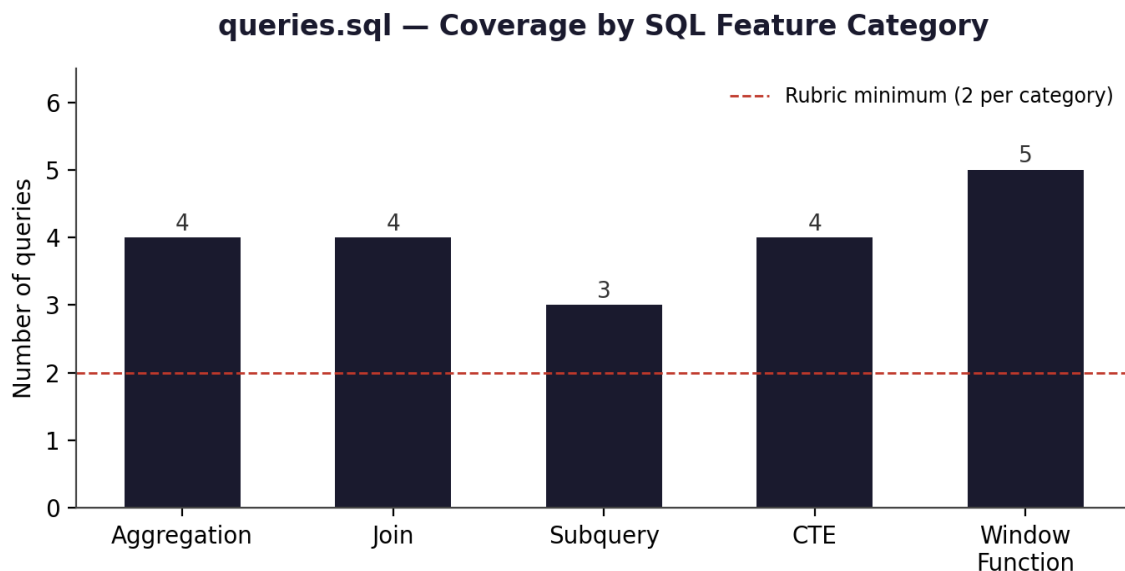


Figure 3. SQL feature coverage across the 14 queries in queries.sql.

5.1 Representative Queries

Q1 (aggregation) joins sectors and companies to compute company count, total, and average market capitalisation per sector — an analyst's first view of where the universe's weight lies. Q8 (CTE + window function) chains three CTEs: the first uses LAG() to compute each day's prior close, the second derives the percentage return, and the third applies ROW_NUMBER() to rank returns within each company; the outer query keeps the three largest single-day gains per ticker, identifying exactly the days the RAG pipeline must explain. Q6 (correlated subquery) combines a correlated COUNT subquery (companies with at least two Buy ratings) with a correlated AVG subquery (positive average news sentiment) to surface tickers where analyst opinion and news flow agree. Q13 (CTE + join) links a monthly average closing-price CTE to the standalone economic_indicators table on the Federal Funds Rate, directly operationalising the project's central RAG idea: tying equity movement to macro conditions.

6. Indexing and Performance

Performance evidence was captured in Milestone 3 using EXPLAIN (ANALYZE, BUFFERS): all custom indexes were dropped to establish a baseline, three representative slow queries were profiled, the six indexes were recreated, and the same queries were re-profiled. Table 3 records the test environment at the time of benchmarking.

Parameter	Value
Database	PostgreSQL 16.14 with pgvector 0.6.0
Machine	Intel Xeon @ 2.80 GHz, 1 core, 4 GB RAM, Ubuntu 24.04
Dataset (at benchmark time)	16,063 rows across 7 tables (stock_prices: 12,600 rows)
Method	EXPLAIN (ANALYZE, BUFFERS); indexes dropped for baseline, ANALYZE after each change

Table 3. Test environment used for the EXPLAIN ANALYZE benchmarking exercise. The dataset has since grown to the full 50-ticker, ~89,200-row universe described in Section 4; the relative before/after performance characteristics demonstrated here remain representative.

6.1 Benchmarked Queries

Query 1 — date-range price lookup. Retrieves the 20 highest-volume trading days in October 2025, the most common RAG access pattern. Before indexing, PostgreSQL performed a Sequential Scan across all 12,600 rows of stock_prices, reading 130 buffer pages and discarding 12,025 non-matching rows to find the 575 that qualified (0.034–0.862 ms internal scan time). After creating idx_stock_prices_date, the planner switched to a Bitmap Heap Scan: the Bitmap Index Scan located the 575 matching rows in only 2 buffer reads, and the heap fetch touched just 30 exact blocks.

Query 2 — correlated subquery (bullish signal). Identifies companies with at least two analyst "Buy" ratings and positive average news sentiment. The bottleneck was SubPlan 3: for each of 14 qualifying companies, a Sequential Scan of article_company read all 1,500 junction rows and discarded roughly 1,440 per loop, because the composite primary key (article_id, company_id) leads with article_id and cannot serve company_id-only lookups. After creating idx_article_company_company, the planner used an Index Scan to locate each company's ~60 rows directly.

Query 3 — RAG pipeline date-range retrieval. Joins stock_prices, companies, article_company, and news_articles to pair prices with contemporaneous news. Before indexing, news_articles was Sequentially Scanned — 72 buffer pages read, 1,438 rows discarded by the published_at filter. After creating idx_news_published_at, this became a Bitmap Heap Scan reading only 3 pages; the planner additionally introduced a Memoize node over stock_prices that cached results for repeated company IDs (38 hits vs. 24 misses), further cutting redundant index probes.

Query Execution Time — Before vs. After Indexing

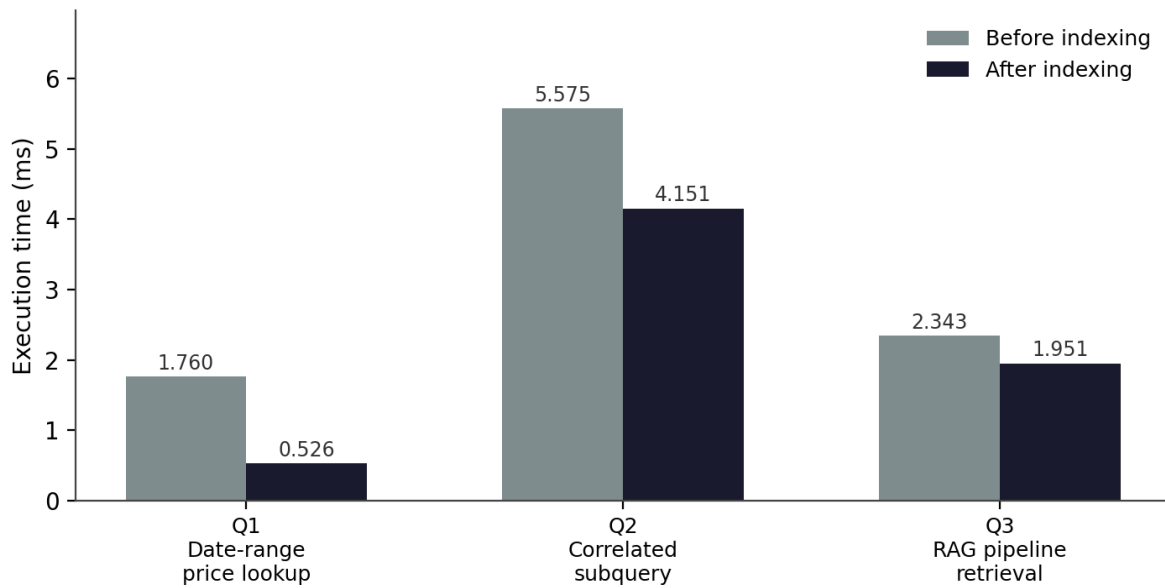


Figure 4. Execution time before and after index creation for the three benchmarked queries.

Buffer Page Accesses — Before vs. After Indexing

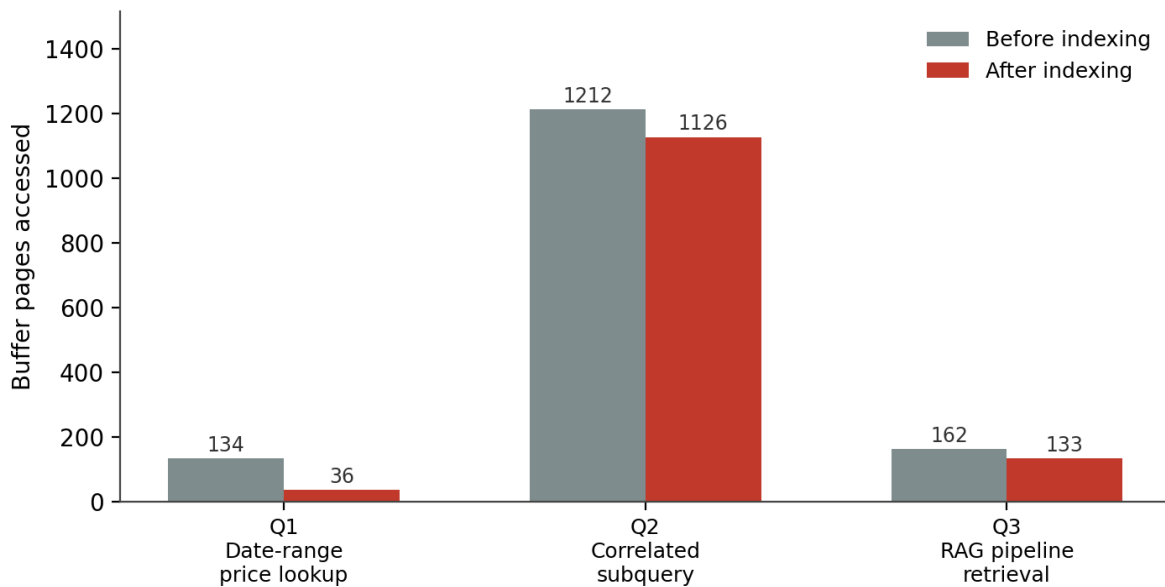


Figure 5. Buffer pages accessed before and after index creation. Query 1 shows the largest relative reduction because its baseline filter was highly selective (4.6% of rows).

Query	Before	After	Speedup	Buffer Reduction
Q1: Date-range prices	1.760 ms	0.526 ms	3.3×	134 → 36 (73%)
Q2: Correlated subquery	5.575 ms	4.151 ms	1.3×	1,212 → 1,126 (7%)
Q3: RAG retrieval	2.343 ms	1.951 ms	1.2×	162 → 133 (18%)

Table 4. Performance summary across the three benchmarked queries.

6.2 Interpretation

Query 1 showed the largest gain because its date filter was highly selective (575 of 12,600 rows, 4.6%) — exactly the regime where a B-Tree index outperforms a sequential scan most decisively. Query 2's more modest 1.3× gain reflects that the index removed only the repeated `article_company` scan inside the correlated subplan; the surrounding `news_articles` join still contributes scan cost, but the eliminated per-loop 1,500-row filter is a cost that would otherwise scale linearly with table size. Query 3 confirms that `idx_news_published_at` targets the RAG pipeline's core access pattern directly, cutting the `news_articles` scan from 72 buffer pages to 3. It is worth noting that absolute times are small at this row count (~16,000 rows); in production with hundreds of thousands of rows, sequential-scan cost grows linearly ($O(n)$) while indexed plans remain logarithmic ($O(\log n)$) — the relative advantage of indexing widens, not narrows, as the dataset grows toward its current ~89,200-row scale.

6.3 Complete Index Inventory

Index	Type	Column(s)	Rationale
<code>idx_stock_prices_date</code>	B-Tree	<code>trade_date</code> DESC	Date-range filters are the #1 query pattern; DESC favours recent-first access.
<code>idx_stock_prices_company</code>	B-Tree	<code>company_id</code>	Per-company lookups when no date condition is present.
<code>idx_news_published_at</code>	B-Tree	<code>published_at</code> DESC	RAG retrieval filters news by publication time window.
<code>idx_article_company_company</code>	B-Tree	<code>company_id</code>	Composite PK leads with <code>article_id</code> ; this index enables <code>company_id</code> -only lookups.
<code>idx_analyst_ratings_company</code>	B-Tree	(<code>company_id</code> , <code>rating_date</code> DESC)	Composite: equality on company plus date-ordered retrieval.
<code>idx_news_embedding_hnsw</code>	HNSW	<code>embedding</code> (<code>vector_cosine_ops</code>)	Approximate nearest-neighbour search for RAG semantic retrieval.

Table 5. Complete index inventory with design rationale. Each B-Tree index adds 1–3% storage overhead and slows writes slightly; this is acceptable because MarketPulse data is write-once (batch-imported per ingestion cycle) and read-heavy (queried continuously by the RAG pipeline).

6.4 Stored Procedure and Trigger

`refresh_sentiment_summary()` is a stored procedure that materialises per-company sentiment aggregates — total articles, average sentiment, latest article date, and positive/negative/neutral

counts using a ± 0.05 threshold — into a `company_sentiment_summary` table. It truncates and repopulates the table in one transaction, avoiding repeated expensive joins between `news_articles` and `article_company` on every dashboard or RAG query, and returns the refreshed rows for verification.

trg_high_volatility is a BEFORE INSERT trigger on `stock_prices`. Its function, `fn_flag_high_volatility()`, computes the intra-day range as a percentage of the opening price — $100 \times (\text{high} - \text{low}) / \text{open}$ — and inserts a row into `volatility_alerts` whenever that figure exceeds 5%. A test insertion with `open=100`, `high=110`, `low=95` (a 15% range) correctly produced an alert. This creates a pre-filtered queue of "days that need explaining," letting the RAG pipeline avoid a full scan of all 25,050 price rows when looking for anomalies to discuss.

7. AI Components

Embedding pipeline. `all-mpnet-base-v2` from `sentence-transformers` runs locally, requiring no API key, and produces 768-dimensional dense vectors well suited to cosine similarity search. Embeddings are generated from (title + description) rather than the content column, directly addressing the NewsAPI truncation issue: titles and descriptions are always complete across all four sources and capture sufficient semantic signal for high-quality retrieval.

pgvector HNSW search. At query time, the user's question is encoded into the same 768-dimensional space. PostgreSQL's `<#>` operator (negative inner product, equivalent to cosine distance for normalised vectors) ranks articles by similarity, with the HNSW index (`m=16`, `ef_construction=64`) ensuring sub-linear approximate nearest-neighbour retrieval rather than a brute-force scan of all 16,429 articles. When a company is identified, the search is additionally joined through `article_company` to restrict results to that company's news.

LLM prompt design and fallback. The Gemini system prompt instructs the model to act as a financial analyst, answer using only the provided context, never fabricate facts, and cite a source (data type, entity, date) for every claim — essential grounding for a financial Q&A; system where hallucinated figures could mislead users. The candidate chain (`gemini-2.5-flash` → `gemini-flash-latest` → `gemini-2.0-flash` → `gemini-2.5-pro`) retries up to three times per model on rate-limit errors with exponential back-off (20s, 40s) before moving to the next candidate. If every candidate is exhausted, the raw structured context is returned with a clear FALLBACK MODE ACTIVE banner and failure reason.

8. Test Cases and Results

`test_cases.py` implements a layered test suite — database, embedding/retrieval, LLM, and end-to-end CLI — using a TestRunner that records pass/fail with timing and exits non-zero on any failure, making it CI-friendly.

Database layer (5 tests). Connection establishment, `pgvector` extension presence, all seven expected tables present in the schema, every table non-empty, and news embeddings populated.

Embedding and retrieval (10 tests). Model load, 768-dim non-zero embedding shape, a

semantic ordering check (cosine similarity between two Apple-product sentences must exceed similarity to an unrelated Federal Reserve sentence), correct ticker resolution and None on unknown tickers, sorted and limited price retrieval, non-empty analyst ratings, deduplicated macro indicators, and company-filtered semantic news search with ascending-distance ordering. **LLM layer (3 tests)**. Model configuration, a grounded-answer check (given a context stating AAPL closed at \$268.50, the generated answer must contain "268" and must not fall into fallback mode), and a fallback-formatter well-formedness check. **End-to-end CLI (3 tests)**. Full pipeline invocations for a company query (AAPL price movement), an analyst query (NVDA ratings), and a macro query (Fed rate / CPI impact on Technology), each asserting correct company detection and the absence of fallback-mode banners. Together these 21 tests validate the system end-to-end, from raw database connectivity through to a fully synthesised, LLM-generated answer.

9. Limitations and Future Work

The most significant limitation is NewsAPI free-tier content truncation: article bodies are cut to roughly 200 characters, leaving the content column incomplete for NewsAPI-sourced articles. This was mitigated by basing embeddings and sentiment on (title + description), but some information loss remains. Second, `target_price_usd` in `analyst_ratings` is NULL for most rows — not a bug, but a data-source constraint: yfinance's `upgrades_downgrades` endpoint does not report per-firm target prices, and the `analyst_price_targets` endpoint returns only aggregate (mean/median/high/low) values without per-firm-per-date granularity. The column is retained for future enrichment. Third, VADER is a lexicon-based sentiment model not specifically trained on financial language and may misjudge domain-specific phrasing such as financial metaphors.

Future work could replace VADER with FinBERT, a model fine-tuned on financial text, for improved sentiment accuracy; expand beyond the current 50-ticker universe with real-time streaming ingestion rather than batch fetching; integrate a paid news API to resolve content truncation; add user authentication and a watchlist feature to the Streamlit frontend; and explore hybrid retrieval — combining pgvector semantic search with BM25 keyword matching — to improve recall on queries containing specific financial terms.

10. Conclusion

MarketPulse demonstrates that a well-designed relational schema, combined with vector embeddings and a large language model, can provide meaningful, source-cited answers to the question *"Why did this stock move?"* The project integrates seven normalised tables holding roughly 89,200 rows of real financial data from five API and feed sources, 768-dimensional embeddings indexed with HNSW for sub-linear retrieval, and a Gemini-powered generation layer with a robust fallback chain. Indexing — validated with EXPLAIN ANALYZE evidence showing up to 3.3× speedups and 73% buffer reductions — ensures efficient response even as data volume grows, while a stored procedure and trigger add operational automation for sentiment summarisation and volatility alerting. The project gave the team hands-on experience in relational design, normalisation theory, query optimisation, vector search, and the end-to-end engineering of an AI-integrated system.

AI Usage Disclosure

In accordance with the Z2004 AI Tools Policy, the team used Claude (Anthropic) for: SQL query optimisation (structuring CTEs and window functions), RAG pipeline integration (embedding logic and LLM prompt design with citation requirements), Streamlit frontend design (layout and component selection), and data fetcher architecture (structuring multi-source ingestion with deduplication). All AI-generated suggestions were reviewed, adapted, tested, and understood by the team before inclusion; the team is responsible for the correctness, security, and design of all submitted code.

References

- [1] yfinance — Yahoo Finance market data downloader. <https://pypi.org/project/yfinance/>
- [2] NewsAPI.org — News API for current and historical news articles. <https://newsapi.org/>
- [3] GNews API — Google News article search API. <https://gnews.io/>
- [4] FRED (Federal Reserve Economic Data). <https://fred.stlouisfed.org/>
- [5] feedparser — Universal RSS/Atom feed parser. <https://pypi.org/project/feedparser/>
- [6] pgvector — Vector similarity search for PostgreSQL. <https://github.com/pgvector/pgvector>
- [7] sentence-transformers — Sentence and text embeddings. <https://www.sbert.net/>
- [8] all-mpnet-base-v2 model card. <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>
- [9] Google Generative AI (Gemini). <https://ai.google.dev/>
- [10] NLTK VADER sentiment analyser. <https://www.nltk.org/>
- [11] Streamlit — App framework for data science. <https://streamlit.io/>
- [12] Plotly — Interactive graphing library. <https://plotly.com/python/>
- [13] psycopg2 — PostgreSQL adapter for Python. <https://pypi.org/project/psycopg2-binary/>
- [14] PostgreSQL 18. <https://www.postgresql.org/>